

The Transported Probability Density Function (PDF) Method and its Relation to the Black-Scholes-Merton Equation

Christian DiPietrantonio

ME 5311: Computational Methods to Viscous Flows
University of Connecticut
May 5, 2026

CONTENTS

I	Introduction	1
II	Financial Derivatives and the Black-Scholes-Merton Equation	1
II-A	Financial Derivatives and Options	1
II-B	The Black-Scholes-Merton Equation	1
III	Derivation of the Black-Scholes-Merton Equation	2
III-A	Stochastic Calculus and Itô's Lemma	2
III-B	The Stock Price Model: Geometric Brownian Motion	3
III-C	Applying Itô's Lemma to the Option Value	3
III-D	Constructing the Hedged Portfolio	3
III-E	No Arbitrage Condition	3
IV	The Transported PDF Method	4
IV-A	Statistical Description of Turbulent Flows	4
IV-B	Eulerian PDF Transport Equation	4
IV-C	The Generalized Langevin Model	4
IV-D	Derivation of the PDF Transport Equation	4
V	The Connection Between the BSM Equation and the Transported PDF Method	5
V-A	Structural Equivalence of the SDEs	5
V-B	Variable Mapping	5
VI	Solution Methods	5
VI-A	Eulerian Methods	5
VI-B	Lagrangian Methods	6
VI-C	Constant vs. Variable Coefficients	6
VII	Computational Experiments and Results	6
VII-A	Problem Description	7
VII-B	Analytical Solution	7
VII-C	Finite Difference Solver (Eulerian)	7
VII-D	Monte Carlo Solver (Lagrangian)	7
VII-E	Results: Option Pricing	8
VII-F	Results: Monte Carlo Visualization and PDF Transport	8
VII-G	Results: Monte Carlo Convergence	9
VIII	Conclusions	10
	References	11
	Appendix	12

I. INTRODUCTION

This paper explores the mathematical connection between the Black-Scholes-Merton (BSM) equation for pricing financial derivatives and the transported probability density function (PDF) method employed in computational fluid dynamics (CFD). The central argument is that these two equations, from completely different fields, are derived from stochastic differential equations (SDEs) with the same drift plus diffusion form and can be solved using the same classes of numerical methods. The BSM equation is derived from the Geometric Brownian Motion SDE, while the transported PDF equation is derived from the generalized Langevin SDE for the velocity of a fluid particle in turbulence. Even though the two equations were developed in separate disciplines, they share a mathematical connection through stochastic calculus.

The paper begins by establishing the financial definitions and background necessary to understand the content of later sections, then derives the BSM equation following the approach outlined by Hull [1], Black and Scholes [2], and Merton [3]. The transported PDF method is then introduced following the work of Pope [4], and its connection with the BSM equation is established through comparison of the underlying SDEs and variable mapping. The paper continues to discuss the solution methods for the BSM and the transported PDF equation, which include grid-based (Eulerian) and particle-based (Lagrangian) approaches. A Python program that implements analytical, Eulerian (finite-difference), and Lagrangian (Monte Carlo) solution methods for pricing a European call option is used to explore these methods in greater detail. The benefits and pitfalls of the different solution approaches are also explored. The paper concludes by discussing the real-world applications of this cross-disciplinary connection and how advancements in either discipline can potentially benefit the other.

II. FINANCIAL DERIVATIVES AND THE BLACK-SCHOLES-MERTON EQUATION

A. Financial Derivatives and Options

In order to understand the Black-Scholes-Merton equation, a foundational understanding of financial derivatives, specifically options, is required. According to Hull, a derivative is a “financial instrument whose value depends on (or derives from) the values of other, more basic, underlying variables” [1]. A stock option, for example, is a derivative whose value is dependent on the price of a stock. However, different derivatives can be dependent on almost any variable, even the weather. For this paper, the primary derivatives of interest are options, which are securities that give “the right to buy or sell an asset, subject to certain conditions, within a specified period of time” [2].

The two major types of options are call and put options. A call option gives the holder the right, but not the obligation, to buy the underlying asset by a specified date at a specified price. A put option gives the holder the right, but not the obligation, to sell the underlying asset by a specified date for a specified price [1]. Options can also be further classified by when they can be exercised. American options can be exercised at any time up to the expiration date as specified in the contract, while European options can only be exercised on the expiration date itself [2]. The price at which the asset may be bought or sold is referred to as the strike price, K , and the expiration date can also be referred to as the maturity date, T .

For the remainder of this paper, the focus will be on European call options. The value of a European call option at expiry can be determined by the following payoff function:

$$V = \max(S_T - K, 0) \quad (1)$$

where S_T is the stock price at expiry. From this payoff function, one can observe that, if the stock price exceeds the strike price at expiry, the holder profits. However, if the stock price at expiry is below the strike price, the option expires worthless, as the holder is not obligated to execute the option. In this latter scenario, the holder only loses what they paid for the option. The Black-Scholes-Merton equation aims to determine a price for the contract, before its value is known at expiry.

B. The Black-Scholes-Merton Equation

In deriving their valuation formula, Black and Scholes [2] assume an idealized, frictionless market with continuous trading, no transaction costs, constant interest rates, no dividends, unrestricted short selling,

and the ability to borrow or lend at the risk-free rate. The option is assumed to be a European call option. Furthermore, an arbitrage-free market is assumed, which means that no trading strategy can generate riskless profits. More explicitly, an arbitrage opportunity can be formally defined as “a (self-financing) trading strategy” that starts with zero value and terminates with a positive value. Therefore, a market is arbitrage-free if there are no such arbitrage opportunities [5].

The assumptions outlined above lead to the derivation of the following second-order linear parabolic Partial Differential Equation (PDE) for the price $V(S, t)$ of a European call option. The same equation was independently derived by Robert C. Merton [3]. Black, Scholes, and Merton all used stochastic calculus to obtain the equation. Nevertheless, the equation below is commonly referred to as the Black-Scholes-Merton PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0 \quad (2)$$

where S is the underlying stock price, t is time, r is the risk-free interest rate, and σ is the volatility of the stock, a measure of how much the stock price fluctuates, or in other words, a “measure of our uncertainty about the returns provided by the stock” [1]. This equation was obtained by forming a hedged portfolio that eliminates risk, which requires that its returns equal the risk-free rate in an arbitrage-free market.

It should also be noted that the structure of Equation 2 can be recognized as having the same form as the general transport equation. Here, $\frac{\partial V}{\partial t}$ is the temporal term, $\frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}$ is a diffusion term, $rS \frac{\partial V}{\partial S}$ is an advection term, and rV is a source (in this case, decay) term. This course has dedicated ample time to discretizing and computationally solving equations of this exact form.

III. DERIVATION OF THE BLACK-SCHOLES-MERTON EQUATION

The use of Stochastic Calculus in the derivation of Equation 2 is crucial to the argument of this paper. Therefore, this section will highlight the key steps of the derivation as shown by Hull [1], Black and Scholes [2], and Merton [3], where stochastic calculus is prevalent.

A. Stochastic Calculus and Itô's Lemma

Before completing the derivation, it is important to highlight a few concepts from Stochastic Calculus that are used in the derivation. First, it should be noted that Stochastic Calculus is a branch of mathematics built specifically to handle the derivatives and integrals of stochastic processes, which are by definition, “an equation describing the probabilistic behavior of a stochastic variable” [1]. Furthermore, a stochastic variable can be defined as a variable “whose future is uncertain”, or in other words, random. The first process to consider is the generalized Wiener process:

$$dx = a dt + b dz \quad (3)$$

where $dz = \epsilon \sqrt{\Delta t}$ is referred to as a Wiener process, and ϵ has a standard normal distribution. Further, an Itô process is a generalized Wiener process where the parameters a and b are functions of the underlying variable and time, and can be expressed as:

$$dx = a(x, t)dt + b(x, t)dz \quad (4)$$

In standard calculus, the chain rule can be used to find the derivative or differential of a multivariable function. However, in Stochastic Calculus, one must use Itô's Lemma to compute the stochastic differential of a function that depends on a stochastic process. Doing so accounts for the fact that second-order terms can not be ignored in the formulation of the differential. This is because the diffusion term in a stochastic process (at least the ones within the scope of this paper) depends on a Wiener Process $dz \sim \sqrt{\Delta t}$, and when squared, this term is on the order of dt . Itô's lemma shows that for a function $G(x, t)$, where x is a stochastic variable following an Itô process, the differential of G can be expressed as

$$dG = \frac{\partial G}{\partial x} dx + \frac{\partial G}{\partial t} dt + \frac{1}{2} \frac{\partial^2 G}{\partial x^2} b^2 dt \quad (5)$$

Again, the key difference from ordinary calculus is the presence of the second-order term $\frac{\partial^2 G}{\partial x^2} b^2$ [1].

B. The Stock Price Model: Geometric Brownian Motion

Returning to the topics of this paper, the price of a stock S is modeled as a stochastic process using the following stochastic differential equation:

$$dS = \mu S dt + \sigma S dW \quad (6)$$

where μ is the stock's expected rate of return (under risk-neutral valuation, $\mu = r$, the risk-free rate), σ is the volatility of the stock price, and dW is a Wiener process (analogous to dz) [1], [6]. This model is commonly referred to as geometric Brownian motion. The first term, $\mu S dt$, represents the deterministic drift of the stock's price, and the second term, $\sigma S dW$, represents random fluctuations. For the remainder of this derivation and paper, μ is replaced with r under risk-neutral valuation [1].

C. Applying Itô's Lemma to the Option Value

Suppose $V(S, t)$ is the value of a European call option dependent on the price of stock S . Given that the price of S is modeled using the Itô process above, Itô's lemma (Equation 5) can be applied, and the result can be rearranged using Equation 6 to obtain [1]:

$$dV = \left(rS \frac{\partial V}{\partial S} + \frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt + \frac{\partial V}{\partial S} \sigma S dW \quad (7)$$

Since the Wiener processes in Equations 6 and 7 are the same, a portfolio consisting of positions in both the option and the stock can be created such that the Wiener process is eliminated [1].

D. Constructing the Hedged Portfolio

This portfolio, as created by Hull, consists of a short position (selling) in one derivative and a long position (buying) in $\frac{\partial V}{\partial S}$ shares of the underlying stock [1].

$$\Pi = -V + \frac{\partial V}{\partial S} S \quad (8)$$

The change in this portfolio over a small time interval, dt , can be expressed as:

$$d\Pi = -dV + \frac{\partial V}{\partial S} dS \quad (9)$$

The expressions above for dV (Equation 7) and dS (Equation 6) can be substituted into the equation above to obtain:

$$d\Pi = \left(-\frac{\partial V}{\partial t} - \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt \quad (10)$$

As expected, the Wiener processes cancel, eliminating randomness from the expression.

E. No Arbitrage Condition

Hull states that because this expression is now free of the randomness induced by the Wiener process, it must be riskless. Therefore, due to the no-arbitrage assumption, the portfolio must grow with the risk-free rate:

$$d\Pi = r \Pi dt \quad (11)$$

Substituting Equation 8 and Equation 10 then yields:

$$\left(-\frac{\partial V}{\partial t} - \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt = r \left(-V + \frac{\partial V}{\partial S} S \right) dt \quad (12)$$

Then, dividing both sides by $-dt$ and rearranging yields the Black-Scholes Merton PDE exactly as

seen in Equation 2:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0 \quad (13)$$

IV. THE TRANSPORTED PDF METHOD

A. Statistical Description of Turbulent Flows

In *Turbulent Flows*, Stephen B. Pope remarks that “in a turbulent flow, the velocity field $U(x, t)$ is random” [4]. This means that if one were to repeat the same experiment multiple times under the same set of conditions, a given event may, but need not, occur, making the event inherently unpredictable. Although a random variable, such as velocity, can not be predicted, the probability of events (e.g., the velocity is less than 10 m/s) can be used to create a Probability Density Function (PDF) that characterizes the random variable. Formally, the probability of any event can be determined by the Cumulative Distribution Function, and the PDF is defined to be the derivative of the CDF. Therefore, according to Pope, a random variable such as the velocity of a turbulent fluid is completely characterized by its PDF, which can be thought of as the probability per unit distance in the sample space [4].

B. Eulerian PDF Transport Equation

The evolution of this Eulerian PDF of velocity in space and time can be represented by a transport equation derived by Pope from the Navier-Stokes equations. The result is the following Eulerian PDF transport equation [4]:

$$\frac{\partial f}{\partial t} + V_i \frac{\partial f}{\partial x_i} = - \frac{\partial}{\partial V_i} \left[f \left\langle \frac{DU_i}{Dt} \middle| \mathbf{V} \right\rangle \right] \quad (14)$$

where $f(\mathbf{V}; \mathbf{x}, t)$ is the Eulerian PDF of velocity.

C. The Generalized Langevin Model

To model the evolution of fluid particle velocities, Pope introduced the generalized Langevin model for a diffusion process. The Langevin equation was originally proposed in 1908 as a stochastic model for the velocity of a microscopic particle undergoing Brownian motion. Pope states that the equation provides a good model for the velocity of a fluid particle in turbulence [4]. The generalized Langevin model used to express the velocity of a fluid particle is expressed below:

$$d\mathbf{U}^*(t) = \mathbf{a}(\mathbf{U}^*(t), \mathbf{X}^*(t), t) dt + b(\mathbf{X}^*(t), t) d\mathbf{W} \quad (15)$$

where $\mathbf{a}(\mathbf{U}^*, \mathbf{X}^*, t)$ is the drift coefficient, $b(\mathbf{X}^*, t)$ is the diffusion coefficient, and $d\mathbf{W}$ is a vector-valued Wiener process [4]. The drift coefficient effectively describes the direction and speed of the system if no randomness were present, and the diffusion coefficient determines the intensity of the random fluctuations induced by the vector-valued Wiener process [4]. According to Pope, the Langevin equation is the simplest stochastic Lagrangian model for a turbulent fluid particle. It should also be noted that the stochastic process $\mathbf{U}^*(t)$ is referred to in the text as the Ornstein-Uhlenbeck (OU) process, and although its PDF is already known to evolve by the Fokker-Planck equation [4], Pope still highlights this transition.

D. Derivation of the PDF Transport Equation

From the Langevin SDE (Equation 15), Itô calculus can be used to derive the Fokker-Planck equation, which is also known as the forward Kolmogorov equation, and describes the transport of the joint Lagrangian PDF of the particle velocity and position, $f_L^*(\mathbf{V}, \mathbf{x}; t)$. The Fokker-Planck equation can then be divided by the marginal position PDF, $f_X^*(\mathbf{x}, t)$, to obtain an expression for the evolution of the conditional PDF of the particle velocity, $f^*(\mathbf{V}|\mathbf{x}, t)$ [4].

$$\frac{\partial f^*}{\partial t} + V_i \frac{\partial f^*}{\partial x_i} = - \frac{\partial}{\partial V_i} [f^* a_i(\mathbf{V}, \mathbf{x}, t)] + \frac{1}{2} b(\mathbf{x}, t)^2 \frac{\partial^2 f^*}{\partial V_i \partial V_i} \quad (16)$$

Pope notes that the conditional PDF, f^* , is the corresponding representation of the Eulerian PDF of velocity, f , for a particle system. Therefore, Equation 16 can be thought of as the Lagrangian equivalent

of the Eulerian PDF transport equation, Equation 14. As a result, Equation 16 is commonly referred to as the Lagrangian PDF transport equation, and again, it describes the transport of the conditional PDF of the particle velocity [4].

V. THE CONNECTION BETWEEN THE BSM EQUATION AND THE TRANSPORTED PDF METHOD

A. Structural Equivalence of the SDEs

Both the Black-Scholes-Merton and Lagrangian PDF Transport equations are derived from stochastic differential equations of the same drift plus diffusion form. Comparing the geometric Brownian motion SDE (Equation 6) with the generalized Langevin SDE (Equation 15) reveals that both equations have the same mathematical structure: deterministic drift term multiplied by dt , plus a diffusion term (random fluctuations) driven by a Wiener process, dW . Furthermore, both PDEs can be derived from their respective SDEs using the same mathematical tools, Stochastic Calculus and Itô's lemma.

B. Variable Mapping

In short, the structural equivalence between the two mathematical frameworks can be summarized through the direct variable mapping:

TABLE I
VARIABLE MAPPING BETWEEN BLACK-SCHOLES-MERTON AND PDF TRANSPORT FRAMEWORKS

Quantitative Finance (GBM / BSM)	CFD (Langevin / PDF Transport)
Stock price, S	Particle velocity, U^*
Risk-free rate, r	Drift Coefficient, a
Volatility, σ	Diffusion coefficient, b
Wiener process, dW	Vector-valued Wiener process, $d\mathbf{W}$
Monte Carlo simulation	Lagrangian particle method

However, a few key distinctions should be made. The BSM equation computes the conditional expectation of a future payoff given the current state (using the PDF of the stock price at that future time), then discounts it back to give the current option price, whereas the Lagrangian Transported PDF equation describes how the PDF of velocity evolves over time. Additionally, the drift and diffusion coefficients (r and σ) are constant for the Brownian Motion SDE, while the coefficients in the Langevin SDE (a and b) depend on the particle's velocity and position at a given time. This has a significant impact on the available solution methods and validation techniques, which are discussed in Section VI. Finally, it should be mentioned that the stochastic differential equation for the velocity of a particle uses a vector-valued Wiener process, because the stochastic perturbations act in multiple spatial directions (typically three-dimensional space). Nevertheless, the variable mapping above establishes a significant connection between the two frameworks and their solution methods.

VI. SOLUTION METHODS

Both the Black-Scholes-Merton and Lagrangian Transported PDF equations can be solved using Eulerian (grid-based) and Lagrangian (particle-based) methods.

A. Eulerian Methods

The Eulerian approach solves the partial differential equation directly on a fixed grid. While Eulerian methods are efficient in low dimensions and provide a deterministic solution, they scale poorly to high dimensions. For example, in order to simulate the price of a portfolio of two stocks, if each stock's price axis was discretized using 500 points, the solver would require 250,000 grid locations to cover all possible combinations. The same concept can be applied to the transported PDF method for high-dimensional PDFs. This is precisely why, when referring to the velocity-frequency joint PDF, Pope states that it is not feasible, computationally, to represent the joint PDF accurately through a discretization of

the seven-dimensional V - θ - x space, as is required in finite-difference, finite-volume, and finite-element methods [4]. Therefore, the transported PDF equations are modeled using Lagrangian methods.

B. Lagrangian Methods

The Lagrangian approach simulates random particle trajectories using the respective underlying stochastic differential equation, then uses statistics to obtain the desired information from the resulting data. The Lagrangian approach most typically used to solve the Black-Scholes-Merton equation is the Monte Carlo method. For this, many random stock price paths are simulated using the Geometric Brownian Motion SDE. The payoff is computed for each path, and the option value is estimated as the average payoff discounted back to the current time. For the transported PDF method, the Lagrangian particle method is used. In this case, the velocities of many fluid particles in a turbulent flow are simulated using the Langevin SDE, and the resulting PDF and its transport are constructed from the particle statistics [4].

The primary advantage of Lagrangian methods is that they scale well with high dimensions. Adding additional state variables (e.g., more stocks or velocity components) just means longer vectors are simulated. In this case, the computational cost grows linearly with added dimensions, whereas with Eulerian methods it grows exponentially. Furthermore, since each path is independent from one another, Lagrangian methods are more compatible with GPUs and High Performance Clusters, which can parallelize computations across many processing units. The primary limitation of Lagrangian models is statistical noise. The standard error of a Monte Carlo analysis is proportional to $1/\sqrt{N}$, where N is the number of simulated paths [1]. Therefore, to cut the error in half, four times as many paths are needed, and to reduce the error by a factor of ten, 100 times as many paths are required.

C. Constant vs. Variable Coefficients

As mentioned previously, the coefficients of the Geometric Brownian Motion and Langevin SDEs have a significant impact on the available solution methods. The constant coefficients of the Geometric Brownian Motion SDE allow for Itô's lemma to be applied to $\ln(S)$, which shows that $\ln(S)$ follows a generalized Wiener process, and further that $\ln(S_T)$ is normally distributed [1]. This is significant for two reasons. First, because $\ln(S_t)$ is normally distributed, then S_t must be lognormally distributed. Therefore, the standard equation for the probability density function of a lognormal variable can be used [7].

$$f(S_t|t) = \frac{1}{S_t \omega \sqrt{2\pi}} \exp \left[-\frac{(\ln(S_t) - \theta)^2}{2\omega^2} \right] \quad (17)$$

where $\theta(t) = \ln(S_0) + (r - \sigma^2/2)t$ and $\omega(t) = \sigma\sqrt{t}$ are the mean and standard deviation of $\ln(S)$ respectively, and t is the current time step. Second, it allows for an exact step formula for the SDE to be obtained [1].

$$S(t + \Delta t) = S(t) \cdot \exp \left[\left(r - \frac{1}{2}\sigma^2 \right) \Delta t + \sigma\epsilon\sqrt{\Delta t} \right] \quad (18)$$

where ϵ is a random sample from a normal distribution with a mean of 0 and a standard deviation of 1.0. For the Langevin SDE, because the coefficients $\mathbf{a}$ and $\mathbf{b}$ depend on the particle's velocity and position at a given time, the Langevin SDE must be discretized directly:

$$\mathbf{U}^*(t + \Delta t) - \mathbf{U}^*(t) = \mathbf{a}(\mathbf{U}^*, \mathbf{X}^*, t) \Delta t + \mathbf{b}(\mathbf{X}^*, t) \Delta \mathbf{W} \quad (19)$$

Furthermore, the standard equation for the probability density function of a lognormal variable can no longer be utilized, and the PDF must be constructed using simulated particle data.

VII. COMPUTATIONAL EXPERIMENTS AND RESULTS

To further demonstrate the connection between the BSM equation and the transported PDF method, and to explore the Eulerian and Lagrangian solution methods, a Python script was developed that utilizes three methods for the BSM equation.

A. Problem Description

The script uses the following parameters for all implemented solution methods:

TABLE II
PARAMETERS USED FOR EUROPEAN CALL OPTION PRICING.

Parameter	Symbol	Value
Strike price	K	\$100
Risk-free rate	r	5%
Volatility	σ	20%
Time to expiry	T	1 year

B. Analytical Solution

The analytical solution to the Black-Scholes formula for a European call option is [1], [2]:

$$V = S N(d_1) - K e^{-r(T-t)} N(d_2) \quad (20)$$

where N is the standard normal cumulative distribution function and:

$$d_1 = \frac{\ln(S/K) + (r + \frac{1}{2}\sigma^2)(T-t)}{\sigma\sqrt{T-t}}, \quad d_2 = \frac{\ln(S/K) + (r - \frac{1}{2}\sigma^2)(T-t)}{\sigma\sqrt{T-t}} \quad (21)$$

where t is the current time. This formula provides the exact solution and can be used to validate the other numerical methods outlined in this paper.

C. Finite Difference Solver (Eulerian)

The BSM PDE (Equation 2) was discretized using an implicit finite difference scheme. The stock price axis was discretized into a uniform grid with 500 intervals from $S = \$0$ to $S_{\max} = \$500$, and time was discretized into 5000 steps marching backwards from $t = T$ to $t = 0$. Second-order central difference stencils were used for the first and second spatial derivatives, and a forward difference was used for the temporal term.

After substituting the stencils above into the PDE and writing $S_j = j\Delta S$, a tridiagonal system of equations is obtained for each time step. This tridiagonal system has the general form of:

$$a_j V_{j-1}^i + b_j V_j^i + c_j V_{j+1}^i = V_j^{i+1} \quad (22)$$

where the coefficients a_j , b_j , and c_j are expressed as:

$$a_j = \frac{\Delta t}{2} (rj - \sigma^2 j^2), \quad b_j = 1 + \Delta t (\sigma^2 j^2 + r), \quad c_j = -\frac{\Delta t}{2} (\sigma^2 j^2 + rj) \quad (23)$$

It should be noted that since this scheme marches backwards in time and the spatial terms are evaluated at the unknown time step (i), it is implicit and unconditionally stable. The terminal condition is the payoff function $V_j^N = \max(j\Delta S - K, 0)$, the boundary condition at the bottom bound of the stock price axis is $V(0, t) = 0$, and the boundary condition at the top bound of the stock price axis is $V(S_{\max}, t) = S_{\max} - K e^{-r(T-t)}$. For this example, with $S_{\max} = \$500$, the strike price $K = \$100$ is roughly 8 standard deviations below the mean of the stock price distribution in log space. Since the probability of this event (the stock price dropping below the strike price) is essentially zero, it is assumed that the option will be exercised at expiry, and the boundary condition $V(S_{\max}, t) = S_{\max} - K e^{-r(T-t)}$ is valid. The tridiagonal system was solved at each time step using a banded solver, producing the option value at each grid point at $t = 0$.

D. Monte Carlo Solver (Lagrangian)

The Monte Carlo simulation was implemented using the exact step formula for the Geometric Brownian motion SDE (Equation 18). 200,000 random paths were simulated to the final time T for 80 stock prices between \$50 and \$200. The value of the option at $t = 0$ can then be obtained using the risk-neutral

valuation formula, by discounting the expected (average) payoff at maturity back to the current time step at the risk-free rate [1].

$$V = e^{-rT} \hat{E} [\max(S_T - K, 0)] \quad (24)$$

E. Results: Option Pricing

Figure 1 shows the option value V as a function of the current stock price S for all three solution methods. The analytical Black-Scholes formula (solid line), the finite difference solver (red squares), and the Monte Carlo simulation (blue circles with error bars) all produce identical results, reinforcing the argument that Eulerian and Lagrangian solution methods produce consistent, accurate results.

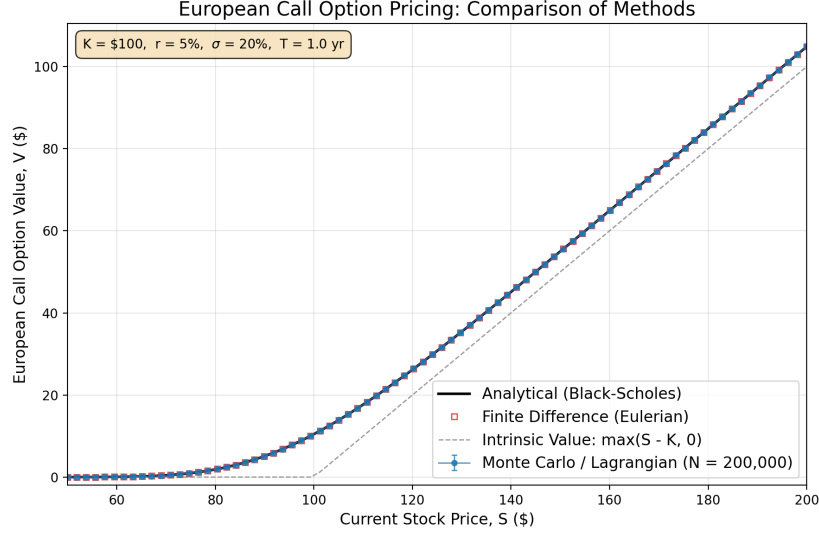


Fig. 1. European call option value vs. stock price: comparison of analytical, Eulerian, and Lagrangian methods

F. Results: Monte Carlo Visualization and PDF Transport

To visualize the stochastic nature of the Monte Carlo simulation, Figure 2 shows 30 individual stock price paths sampled from 100 simulated paths, all starting at $S_0 = \$100$. The diffusion of the paths over time demonstrates the effect of volatility. The bold blue line represents the mean of the 100 simulated paths, and the black dashed line represents the theoretical expected stock price, $E(S_t) = S_0 e^{rt}$ [1]. As the number of simulated paths increases (and with this the accuracy of the Monte Carlo simulation), the mean converges to this theoretical value.

The data from the larger Monte Carlo simulation (200,000 paths) was then used to construct the probability density function of the stock price at each time step, mirroring the Lagrangian particle method in CFD. For a turbulent flow, particle paths would be simulated using the Langevin SDE, their velocities (conditioned on position and time) would be recorded at each time step, and the PDF surface would be constructed from the statistics. Figure 3 shows the 3D PDF evolution surface for a stock with initial price $S_0 = \$100$. The surface was constructed entirely from Monte Carlo particle data, as the 200,000 simulated paths were used to create a histogram at selected time steps. The PDF starts as a sharp spike near the initial stock price, then shifts and spreads over time due to drift and diffusion.

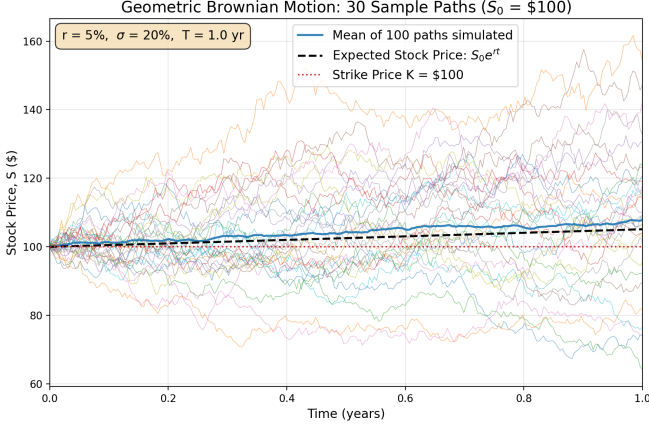


Fig. 2. Geometric Brownian Motion: 30 sample paths with mean and expected return.

PDF Evolution from Monte Carlo Particle Data

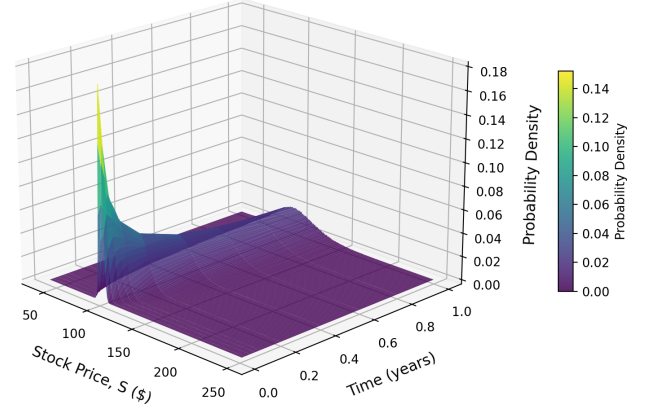


Fig. 3. PDF evolution from Monte Carlo particle data.

Figure 4 provides a clearer view of the drift and diffusion of the PDF. It also further demonstrates the accuracy of the Monte Carlo simulation by comparing its results to the analytical expression for the probability density function of a lognormal variable (Equation 17). The shaded bars are PDFs constructed from Monte Carlo, and the solid curves are the analytical lognormal PDFs; great agreement can be observed between the two. Over time, the volatility spreads the distribution of possible stock prices (diffusion), and the drift shifts it to the right (advection). For a turbulent flow, drift can be thought of as where the average velocity trends, and diffusion as the effect of turbulence randomizing the velocity around the mean. Figure 5 shows a top-down heatmap view of the PDF evolution, with the expected stock price $E(S_t) = S_0 e^{rt}$ and the $\pm 1\sigma$ bounds overlaid. The drift of the PDF is clearly visible, aligning with the expected stock price, which can also be thought of as the deterministic growth of the stock in the absence of volatility.

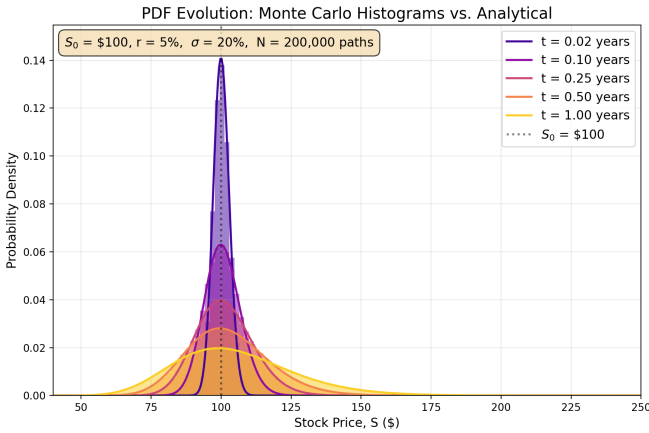


Fig. 4. PDF Evolution: Monte Carlo histograms vs. log-normal PDF.

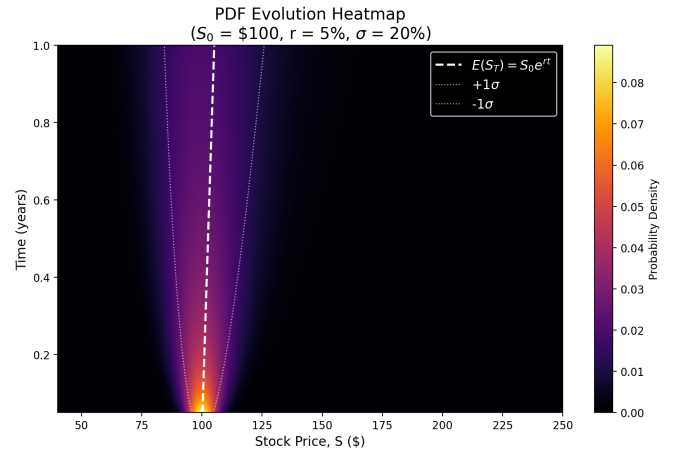


Fig. 5. PDF evolution heatmap with drift path and $\pm 1\sigma$ bounds.

G. Results: Monte Carlo Convergence

Finally, Figure 6 demonstrates the convergence behavior of the Monte Carlo method. The standard error of the estimate produced by the simulation is expressed as $\omega/\sqrt{N}$ [1], where ω is the standard deviation of the discounted payoffs, and N is the number of simulated paths. With 100 paths, the estimate is noisy because each path has a greater weight. However, by 1,000,000 paths, the estimate approaches the exact value obtained using the Black-Scholes formula. As mentioned in Section VI, this slow convergence rate

is a major limitation of Lagrangian approaches.

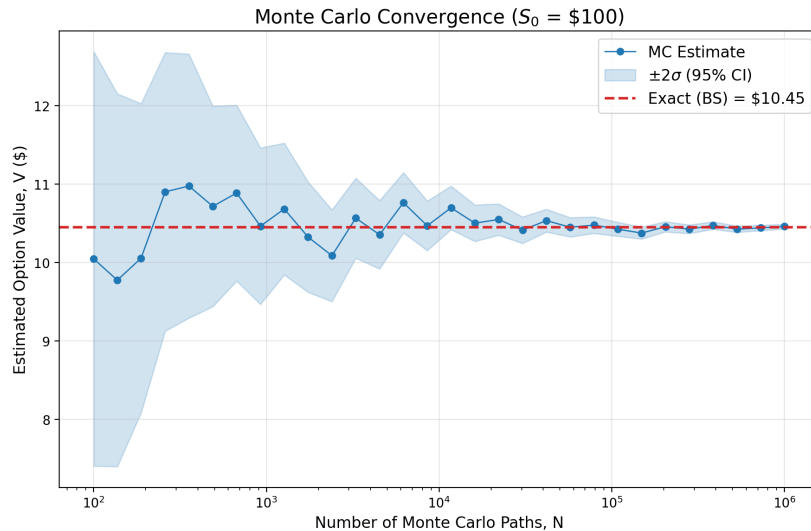


Fig. 6. Monte Carlo convergence: estimated option value vs. number of simulated paths.

VIII. CONCLUSIONS

The contents of this paper and the results of the computational experiments have demonstrated the mathematical connection between the Black-Scholes-Merton equation for pricing financial derivatives and the transported probability density function method used in computational fluid dynamics. From the material presented in this paper, several conclusions can be drawn.

First, the BSM PDE and the Fokker-Planck equation (forward Kolmogorov) are both derived from SDEs of the same drift-diffusion form, establishing a direct mathematical connection between financial derivative pricing and the transported PDF method in CFD.

Second, both Eulerian (finite difference) and Lagrangian (Monte Carlo) methods produce results consistent not only with each other but also with analytical solutions. This was validated by the agreement between the numerical results obtained through the experimental Eulerian and Lagrangian solvers and the exact values obtained using the analytical Black-Scholes pricing formula.

Third, Monte Carlo particle simulations can reproduce results consistent with analytical probability density functions, as demonstrated in Section VII-F and Figure 4. This is an extremely important conclusion in the case of the transported PDF method, where no analytical probability density function exists.

Fourth, in the case of the Black-Scholes-Merton equation, an analytical solution exists for the Geometric Brownian Motion SDE of a stock price, which implies that a stock's price at a future time (given its initial price) is lognormally distributed. As a result, the analytical expression for the probability density function of a lognormal random variable can be used to obtain the PDF. Therefore, the BSM equation can be used to validate Lagrangian particle methods, because the PDFs generated from the particle methods can be compared with those obtained analytically, as demonstrated with the results of the Monte Carlo experiment discussed in this paper.

Finally, advancements in financial modeling and computational fluid dynamics are mutually beneficial. The shared mathematical framework illustrated in this paper means that numerical methods, variance reduction techniques, and other solution strategies developed independently in the fields of either quantitative finance or computational fluid dynamics can possibly be utilized or even directly applied to problems in the other.

REFERENCES

- [1] J. C. Hull, *Options, Futures, and Other Derivatives*, 10th ed. Boston, MA: Pearson, 2022.
- [2] F. Black and M. Scholes, "The pricing of options and corporate liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.
- [3] R. C. Merton, "Theory of rational option pricing," *Bell Journal of Economics and Management Science*, vol. 4, no. 1, pp. 141–183, 1973.
- [4] S. B. Pope, *Turbulent Flows*. Cambridge: Cambridge University Press, 2000.
- [5] M. Baxter and A. Rennie, *Financial Calculus: An Introduction to Derivative Pricing*. Cambridge: Cambridge University Press, 1996.
- [6] S. E. Shreve, *Stochastic Calculus for Finance II: Continuous-Time Models*. New York, NY: Springer, 2004.
- [7] D. C. Montgomery, G. C. Runger, and N. F. Hubele, *Engineering Statistics*, 5th ed. Hoboken, NJ: John Wiley & Sons, 2011.

APPENDIX

```

1  """
2  Christian DiPietrantonio
3  ME 5311: Computational Methods to Viscous Flows
4  Term Paper
5  04/19/2026
6  -----
7  Black-Scholes-Merton Solver
8  -----
9
10 This script demonstrates three approaches to pricing a European call option:
11     1. Analytical Black-Scholes formula (closed-form solution)
12     2. Monte Carlo simulation (Lagrangian / stochastic particle method)
13     3. Finite Difference (Eulerian / grid-based PDE solver)
14
15 Methods 2 and 3 are the Lagrangian and Eulerian numerical approaches.
16 """
17
18 import numpy as np
19 from scipy.stats import norm
20 from scipy.linalg import solve_banded
21 import matplotlib.pyplot as plt
22 from matplotlib import cm
23 from mpl_toolkits.mplot3d import Axes3D
24 import os
25
26 # -----
27 # Parameters
28 # -----
29 K      = 100.0 # Strike Price ($)
30 r      = 0.05 # Risk-free interest rate (5%)
31 sigma  = 0.2 # Volatility
32 T      = 1.0 # Time to expiry (1 year)
33
34 # Monte Carlo Settings
35 N_paths = 200000 # Number of simulated paths
36 N_steps = 252    # Time steps (trading days in a year)
37 N_S_pts = 80     # Number of starting prices to evaluate
38
39
40 # Stock price range
41 S_min = 50
42 S_max = 200
43
44 # Output directory
45 out_dir = "figures"
46 os.makedirs(out_dir, exist_ok=True)
47
48 # -----
49 # Analytical Black-Scholes Formula
50 # -----
51 def bs_call(S, K, r, sigma, T):
52     """
53     Analytical Black-Scholes price for a European call option.
54     Page 644, Black-Scholes
55     Page 91, Baxter-Rennie
56     Page 336, Hull
57
58     Parameters
59     -----
60     S      : current stock price
61     K      : strike price
62     r      : risk-free rate
63     sigma  : volatility
64     T      : time to expiration

```

```

65
66 Returns
67 -----
68 V      : option prices
69 """
70
71 d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
72 d2 = d1 - sigma * np.sqrt(T)
73 V = S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
74
75 return V
76
77 # -----
78 # Monte Carlo Solver - Terminal Value Method
79 # -----
80 def mc_option_price(S0, K, r, sigma, T, N_paths):
81     """
82     Price a European call via Monte Carlo using the exact GBM solution. Instead of stepping
83     through time, we jump directly to S(T). This works because we only need the terminal
84     stock price to compute the payoff. For the PDF evolution, we must use the full time-
85     stepping version below.
86
87     Page 217, Hull (Long position European Call option payoff)
88     Page 336, Hull (Option expected value discounted at risk-free rate)
89     Page 471, Hull (Stock price at expiration time)
90     Page 473, Hull (Standard Error of estimate)
91
92     Parameters
93     -----
94     S0      : starting stock price
95     K       : strike price
96     r       : risk free rate
97     sigma   : volatility
98     T       : time to expiration
99     N_paths : number of simulated paths
100
101     Returns
102     -----
103     V_mean : estimated option price/value
104     V_std  : standard error of the estimate
105
106     """
107
108     Z = np.random.standard_normal(N_paths)
109     ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z)
110
111     # payoff for a European call
112     payoffs = np.maximum(ST - K, 0.0)
113
114     # discount back to present value
115     V_mean = np.exp(-r * T) * np.mean(payoffs)
116     V_std = np.exp(-r * T) * np.std(payoffs) / np.sqrt(N_paths)
117
118     return V_mean, V_std
119
120 # -----
121 # Monte Carlo Solver - Full Path Simulation
122 # -----
123 def mc_full_paths(S0, r, sigma, T, N_paths, N_steps):
124     """
125     Simulate full GBM paths.
126
127     Page 470, Hull (Discretizing SDEs)
128     Note:

```

```

127     For this example, we use the more accurate analytical solution for the GBM SDE. For the
128         transported PDF method, we would discretize the SDE itself.
129 Page 471, Hull (Stock price at time step)
130
131 Parameters
132 -----
133 S0      : starting stock price
134 r       : risk-free rate
135 sigma   : volatility
136 T       : time to expiration
137 N_paths : number of simulated paths
138 N_steps : number of time steps
139
140 Returns
141 -----
142 S       : all price paths
143 t       : time array
144
145 """
146
147 dt = T / N_steps
148 t = np.linspace(0, T, N_steps + 1)
149
150 S = np.zeros((N_steps + 1, N_paths))
151 S[0, :] = S0
152
153 # generate random increments
154 Z = np.random.standard_normal((N_steps, N_paths))
155
156 # step through time
157 for i in range(N_steps):
158     S[i+1, :] = S[i, :] * np.exp((r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z[i, :])
159
160 return S, t
161
162 # -----
163 # Finite Difference Solver
164 # -----
165 def fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000):
166     """
167     Solves the BSM PDE using implicit finite difference method.
168
169     This is the Eulerian approach: discretize the stock price into a grid and solve the PDE
170         directly at each point. Important to note that we march backwards in time from t = T (
171         expiry) to t=0 (today).
172
173     Parameters
174     -----
175     K      : strike price
176     r      : risk-free rate
177     sigma   : volatility
178     T      : time to expiration
179     S_max   : upper bound of stock price grid
180     M      : number of spatial intervals
181     N_t     : number of time steps
182
183     Returns
184     -----
185     S_grid : stock price at grid points
186     V      : option values at t = 0 (today)
187     """
188
189     # ----- Grid Setup -----
190     dS = S_max / M      # spatial step size
191     dt = T / N_t        # time step size

```



```

189 S_grid = np.linspace(0, S_max, M + 1)
190
191 # ----- Terminal Condition -----
192 # At expiry, option value = payoff = max(S - K, 0)
193 V = np.maximum(S_grid - K, 0.0)
194
195 # ----- Build Tridiagonal Coefficients for Interior Nodes j = 1, ..., M - 1 -----
196 j = np.arange(1, M)
197
198 a_j = 0.5 * dt * (r * j - sigma**2 * j**2)
199 b_j = 1.0 + dt * (sigma**2 * j**2 + r)
200 c_j = -0.5 * dt * (sigma**2 * j**2 + r * j)
201
202 # ----- Format into Banded System -----
203 n_interior = M - 1 # number of unknowns
204 # Banded matrix (l + u + 1, n) where l = 1 lower diagonal, u = 1 upper diagonal
205 # element[row, col] maps to ab[u + row - col, col]
206 ab = np.zeros((3, n_interior))
207 ab[0, 1:] = c_j[:-1]
208 ab[1, :] = b_j
209 ab[2, :-1] = a_j[1:]
210
211 # ----- Time Marching -----
212 for n in range(N_t):
213     rhs = V[1:M].copy()
214
215     time_to_expiry = (n + 1) * dt
216     V_top = S_max - K * np.exp(-r * time_to_expiry)
217
218     rhs[-1] -= c_j[-1] * V_top
219
220     # Solve the tridiagonal System
221     V[1:M] = solve_banded((1, 1), ab, rhs)
222
223     # Apply boundary conditions to full solution array
224     V[0] = 0.0
225     V[M] = V_top
226
227 return S_grid, V
228
229 # -----
230 # Figure 1: V(S) Comparison
231 # -----
232 def plot_option_value_curve():
233     """
234     Compare Monte Carlo, Finite Difference, and Analytical Option Prices.
235     """
236
237     print("Computing V(S) curve ...")
238     S_arr = np.linspace(S_min, S_max, N_S_pts)
239
240     #Analytical Solution
241     V_analytical = bs_call(S_arr, K, r, sigma, T)
242
243     # Monte Carlo (loop over starting prices)
244     V_mc = np.zeros(N_S_pts)
245     V_err = np.zeros(N_S_pts)
246
247     for i, S0 in enumerate(S_arr):
248         V_mc[i], V_err[i] = mc_option_price(S0, K, r, sigma, T, N_paths)
249
250     # Finite Difference
251     print(" Running finite difference solver...")
252     S_fd, V_fd = fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000)
253

```

```

254 # Interpolate FD results onto same S_arr for comparison
255 V_fd_interp = np.interp(S_arr, S_fd, V_fd)
256
257 # ----- Plot -----
258 fig, ax = plt.subplots(figsize=(9, 6))
259
260 # Analytical Solution
261 ax.plot(S_arr, V_analytical, 'k-', linewidth = 2, label='Analytical (Black-Scholes)')
262
263 # Finite Difference Solution
264 ax.plot(S_arr, V_fd_interp, 's', color='tab:red', markersize=4, markerfacecolor='none',
265         markedgewidth=1.0, label='Finite Difference (Eulerian)', alpha=0.8)
266
267 # Monte Carlo (with error bars)
268 ax.errorbar(S_arr, V_mc, yerr=4*V_err, marker='o', color='tab:blue', markersize=4, linewidth
269             =1.0, capsize=2, label=f'Monte Carlo / Lagrangian (N = {N_paths:,})', alpha=0.8)
270
271 # Intrinsic Value for Reference
272 intrinsic = np.maximum(S_arr - K, 0)
273 ax.plot(S_arr, intrinsic, '--', color='tab:gray', linewidth=1.0, alpha=0.8, label='Intrinsic
274         Value: max(S - K, 0)')
275
276 ax.set_xlabel('Current Stock Price, S ($)', fontsize=12)
277 ax.set_ylabel('European Call Option Value, V ($)', fontsize=12)
278 ax.set_title('European Call Option Pricing: Comparison of Methods', fontsize=15)
279 ax.legend(fontsize=12)
280 ax.set_xlim(S_min, S_max)
281 ax.set_ylim(bottom=-2)
282 ax.grid(True, alpha=0.3)
283
284 param_text = (fr'K = \${K:.0f}, r = {r:.0%}, \sigma = {\sigma:.0%}, T = {T:.1f} yr')
285 ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=10, verticalalignment='top',
286         , bbox=dict(boxstyle='round', facecolor='wheat', edgecolor='black', pad=0.5, alpha=0.8))
287
288 fig.tight_layout()
289 fig.savefig(f'{out_dir}/figure01_option_value_curve.png', dpi=200, bbox_inches="tight")
290 plt.close(fig)
291 print(f'  Saved: {out_dir}/figure01_option_value_curve.png')
292
293 return S_arr, V_analytical, V_mc, V_err, V_fd_interp
294
295 # -----
296 # Figure 2: Sample GBM Paths
297 # -----
298 def plot_sample_paths(S0=100.0, N_show=30, N_sim=100):
299     """
300     Show a handful of GBM trajectories to illustrate stochastic behavior.
301
302     Page 323, 325, Hull (Expected Return)
303     """
304     print("Simulating sample paths...")
305     S, t = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)
306
307     fig, ax = plt.subplots(figsize=(9, 6))
308
309     # plot subset of paths
310     for j in range(N_show):
311         ax.plot(t, S[:, j], linewidth=0.5, alpha=0.6)
312
313     # overlay the mean of the simulated paths
314     S_mean_sim = np.mean(S, axis=1)
315     ax.plot(t, S_mean_sim, color='tab:blue', linewidth=2.0, alpha=0.9, label=f'Mean of {N_sim}
316         paths simulated')

```

```

314 # overlay the expected return
315 S_expected = S0 * np.exp(r * t)
316 ax.plot(t, S_expected, 'k--', linewidth=2.0, label=f'Expected Stock Price:  $S_0 e^{rt}$ ')
317
318 ax.axhline(K, color='tab:red', linestyle=':', linewidth=1.5, label=f'Strike Price K =  $\{K:0$ 
    f}')
319
320 ax.set_xlabel('Time (years)', fontsize=12)
321 ax.set_ylabel('Stock Price, S ($)', fontsize=12)
322 ax.set_title(fr'Geometric Brownian Motion:  $\{N\_show\}$  Sample Paths ( $S_0 = \{S0:0f\}$ )',
    fontsize=15)
323 ax.legend(fontsize=12, loc='upper center', bbox_to_anchor=(0.6, 1.0))
324 ax.set_xlim(0, T)
325 ax.grid(True, alpha=0.3)
326
327 param_text = (fr'r =  $\{r:0\}$ ,  $\sigma = \{\sigma:0\}$ , T =  $\{T:1f\}$  yr')
328 ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=12, verticalalignment='top',
    , bbox=dict(boxstyle='round', facecolor='wheat', edgecolor='black', pad=0.5, alpha=0.8))
329
330 fig.tight_layout()
331 fig.savefig(f'{out_dir}/figure02_GMB_sample_paths.png', dpi=200, bbox_inches="tight")
332 plt.close(fig)
333 print(f' Saved: {out_dir}/figure02_GMB_sample_paths.png')
334
335 return S, t
336
337 # -----
338 # Figure 3: PDF Evolution - 3D Surface
339 # -----
340
341 def plot_pdf_evolution(S0=100.0, N_sim=200000):
342     """
343     Visualize how the probability density of a stock price evolves over time using Monte Carlo
344     particle data.
345     Note:
346         - The exact GBM sol could be used to construct the PDF using the PDF function for a
347           lognormal distribution. However, here the PDF is constructed using MC data, because
348           no PDF function is available for the Langevin SDE.
349
350     Page 322, Hull (Mean and Standard Deviation of  $\ln ST$ )
351     Page 84, Montgomery, Runger, Hubele
352     """
353
354     print("Computing PDF evolution surface from Monte Carlo data...")
355
356     t_snapshots = np.array([0.01, 0.025, 0.05, 0.075, 0.1, 0.2, 0.4, 0.6, 0.8, 1.0])
357     S_bins = np.linspace(40, 250, 500)
358     S_bin_centers = 0.5 * (S_bins[:-1] + S_bins[1:])
359
360     # Simulate paths
361     S_all, t_all = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)
362
363     # Build PDF at each time snapshot
364     PDF_mc = np.zeros((len(t_snapshots), len(S_bin_centers)))
365     for i, t_val in enumerate(t_snapshots):
366         idx = np.argmin(np.abs(t_all - t_val))
367         counts, _ = np.histogram(S_all[idx, :], bins=S_bins, density=True)
368         PDF_mc[i, :] = counts
369
370     fig = plt.figure(figsize=(9, 6))
371     ax = fig.add_subplot(111, projection='3d')
372     T_mesh, S_mesh = np.meshgrid(t_snapshots, S_bin_centers, indexing='ij')
373     surf = ax.plot_surface(S_mesh, T_mesh, PDF_mc, cmap='viridis', alpha=0.85, rstride=1,
374         cstride=3, edgecolor='none')

```

```

372 ax.set_xlabel('Stock Price, S ($)', fontsize=12, labelpad=10)
373 ax.set_ylabel('Time (years)', fontsize=12, labelpad=10)
374 ax.set_zlabel('Probability Density', fontsize=12, labelpad=10)
375 ax.set_title('PDF Evolution from Monte Carlo Particle Data', fontsize=15)
376 ax.view_init(elev=20, azim=-45)
377 fig.colorbar(surf, ax=ax, shrink=0.45, aspect=15, label='Probability Density', pad=0.10)
378
379 fig.tight_layout()
380 fig.savefig(f'{out_dir}/figure03_3d_pdf_evolution.png', dpi=200, bbox_inches="tight")
381 plt.close(fig)
382 print(f'  Saved: {out_dir}/figure03_3d_pdf_evolution.png')
383 return t_snapshots, S_bin_centers, PDF_mc
384
385 # -----
386 # Figure 4: PDF Evolution - 2D Slices
387 # -----
388 def plot_pdf_slices(S0=100.0, N_sim=200000):
389     """
390     Monte Carlo histograms overlaid with analytical log-normal curves. This demonstrates that
391     the Lagrangian statistics converge to the Eulerian PDF.
392
393     Page 322, Hull (Mean and Standard Deviation of ln ST)
394     Page 84, Montgomery, Runger, Hubele
395     """
396
397     print("Computing PDF time slices...")
398
399     t_slices = [0.02, 0.1, 0.25, 0.5, 1.0]
400     S_grid = np.linspace(40, 250, 500)
401
402     # Simulate paths
403     S_all, t_all = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)
404
405     colors = cm.plasma(np.linspace(0.1, 0.9, len(t_slices)))
406
407     fig, ax = plt.subplots(figsize=(9, 6))
408     pdf_max = 0.0
409
410     for i, t_val in enumerate(t_slices):
411         # Monte Carlo Histogram
412         idx = np.argmin(np.abs(t_all - t_val))
413         ax.hist(S_all[idx, :], bins=120, range=(40, 250), density=True, alpha=0.5, color=colors[i],
414                edgecolor='none')
415
416         # Analytical log-normal PDF
417         mu_ln = np.log(S0) + (r - 0.5 * sigma**2) * t_val
418         sigma_ln = sigma * np.sqrt(t_val)
419         pdf_vals = (1.0 / (S_grid * sigma_ln * np.sqrt(2*np.pi)) * np.exp(-0.5 * ((np.log(S_grid)
420                - mu_ln) / sigma_ln)**2))
421         pdf_max = max(pdf_max, np.max(pdf_vals))
422         ax.plot(S_grid, pdf_vals, color=colors[i], linewidth=2.0, label=f't = {t_val:.2f} years')
423
424     ax.axvline(S0, color='black', linestyle=':', linewidth=2.0, alpha=0.5, label=fr'$S_0$ = \${S0:.0f}$')
425
426     ax.set_xlabel('Stock Price, S ($)', fontsize=12)
427     ax.set_ylabel('Probability Density', fontsize=12)
428     ax.set_title('PDF Evolution: Monte Carlo Histograms vs. Analytical', fontsize=15)
429     ax.legend(fontsize=12, loc='upper right')
430     ax.set_xlim(40, 250)
431     ax.set_ylim(0, pdf_max * 1.1)
432     ax.grid(True, alpha=0.3)
433
434     param_text = (fr'$S_0$ = \${S0:.0f}, r = {r:.0%}, $\sigma$ = {sigma:.0%}, N = {N_sim:,}
435                  paths')

```

```

432     ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=12, verticalalignment='top'
433            , bbox=dict(boxstyle='round', facecolor='wheat', edgecolor='black', pad=0.5, alpha=0.8))
434
435     fig.tight_layout()
436     fig.savefig(f'{out_dir}/figure04_2D_pdf_slices.png', dpi=200, bbox_inches="tight")
437     plt.close(fig)
438     print(f'    Saved: {out_dir}/figure04_2D_pdf_slices.png')
439
440     # -----
441     # Figure 5: Monte Carlo Convergence
442     # -----
443     def plot_convergence(S0=100.0):
444         """
445         Show how the MC estimate converges as more paths are simulated.
446
447         Page 473, Hull (MC Standard Error/Confidence intervals)
448         """
449         print("Computing convergence study...")
450         V_exact = bs_call(S0, K, r, sigma, T)
451         N_range = np.logspace(2, 6, 30).astype(int)
452         V_estimates = np.zeros(len(N_range))
453         V_errors = np.zeros(len(N_range))
454
455         np.random.seed(11)
456         for i, N in enumerate(N_range):
457             V_estimates[i], V_errors[i] = mc_option_price(S0, K, r, sigma, T, N)
458
459         fig, ax = plt.subplots(figsize=(9, 6))
460         ax.semilogx(N_range, V_estimates, 'o-', color='tab:blue', markersize=5, linewidth=1.0, label
461                    ='MC Estimate')
462         ax.fill_between(N_range, V_estimates - 2*V_errors, V_estimates + 2*V_errors, alpha=0.2,
463                        color='tab:blue', label=r'$\pm 2 \sigma$ (95% CI)')
464         ax.axhline(V_exact, color='tab:red', linestyle='--', linewidth=2.0, label=f'Exact (BS) = ${
465                        V_exact:.2f}$')
466
467         ax.set_xlabel('Number of Monte Carlo Paths, N', fontsize=12)
468         ax.set_ylabel('Estimated Option Value, V ($)', fontsize=12)
469         ax.set_title(fr'Monte Carlo Convergence ($S_0$ = ${S0:.0f}$)', fontsize=15)
470         ax.legend(fontsize=12)
471         ax.grid(True, alpha=0.3)
472
473         fig.tight_layout()
474         fig.savefig(f'{out_dir}/figure05_MC_convergence.png', dpi=200, bbox_inches="tight")
475         plt.close(fig)
476         print(f'    Saved: {out_dir}/figure05_MC_convergence.png')
477
478     # -----
479     # Figure 6: PDF Evolution - Heatmap
480     # -----
481     def plot_pdf_heatmap(S0=100.0):
482         """
483         2D Heatmap: x = stock price, y = time, color = probability density.
484
485         Page 84, Montgomery, Runger, Hubele (Mean and Variance in Price Space)
486         Page 315, Hull (Mean and Variance in log space)
487         """
488         print("Computing PDF heatmap...")
489         t_grid = np.linspace(0.05, T, 500)
490         S_grid = np.linspace(40, 250, 500)
491
492         PDF = np.zeros((len(t_grid), len(S_grid)))
493         for i, t_val in enumerate(t_grid):
494             mu_ln = np.log(S0) + (r - 0.5 * sigma**2) * t_val

```

```

493     sigma_ln = sigma * np.sqrt(t_val)
494     PDF[i, :] = (1.0 / (S_grid * sigma_ln * np.sqrt(2*np.pi)) * np.exp(-0.5 * ((np.log(S_grid
      ) - mu_ln) / sigma_ln)**2))
495
496 fig, ax = plt.subplots(figsize=(9, 6))
497 S_mesh, t_mesh = np.meshgrid(S_grid, t_grid)
498 pcm = ax.pcolormesh(S_mesh, t_mesh, PDF, cmap='inferno', shading='gouraud')
499
500 S_expected = S0 * np.exp(r * t_grid)
501 ax.plot(S_expected, t_grid, 'w--', linewidth=2.0, label=r'$E(S_T) = S_0 e^{rt}$')
502
503 for sign, lbl in [(1, r'+1$\sigma$'), (-1, r'-1$\sigma$')]:
504     mu_ln = np.log(S0) + (r - 0.5 * sigma**2) * t_grid
505     sigma_ln = sigma * np.sqrt(t_grid)
506     S_bound = np.exp(mu_ln + sign * sigma_ln)
507     ax.plot(S_bound, t_grid, 'w:', linewidth=1.0, label=lbl, alpha=0.7)
508
509 ax.set_xlabel('Stock Price, S ($)', fontsize=12)
510 ax.set_ylabel('Time (years)', fontsize=12)
511 ax.set_title('PDF Evolution Heatmap\n' \
512             fr'($S_0$ = ${S0:.0f}, r = {r:.0%}, $\sigma$ = ${sigma:.0%})', fontsize=15)
513 ax.legend(fontsize=12, loc='upper right', facecolor='black', edgecolor='white', labelcolor='
      white')
514 fig.colorbar(pcm, ax=ax, label='Probability Density')
515 ax.set_xlim(40, 250)
516
517 fig.tight_layout()
518 fig.savefig(f'{out_dir}/figure06_pdf_heatmap.png', dpi=200, bbox_inches="tight")
519 plt.close(fig)
520 print(f'  Saved: {out_dir}/figure06_pdf_heatmap.png')
521
522 # -----
523 # Main Driver
524 # -----
525 def main():
526     print("=" * 120)
527     print("BSM Solver - Monte Carlo + Finite Difference + Analytical")
528     print("=" * 120)
529     print(fr'Parameters: K=${K}, r={r:.2%}, $\sigma$={sigma:.2%}, T={T} years')
530     print(f'Monte Carlo: {N_paths:,} paths, {N_steps} time steps')
531     print("=" * 120)
532
533     np.random.seed(2026)
534
535     S_arr, V_an, V_mc, V_err, F_fd = plot_option_value_curve()
536     S_paths, t_arr = plot_sample_paths()
537     t_snap, S_centers, PDF_mc = plot_pdf_evolution()
538     plot_pdf_slices()
539     plot_convergence()
540     plot_pdf_heatmap()
541
542     print("\n" + "=" * 120)
543     print("V(S) at selected stock prices")
544     print("=" * 120)
545     print(f'{'S ($)':>8} {'Analytical':>12} {'Monte Carlo':>12} {'Finite Difference':>12} {'MC
      Error':>8}')
546     print("-" * 120)
547
548     S_fd_check, V_fd_check = fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000)
549     check_prices = [60, 80, 100, 120, 140, 160]
550     for S0 in check_prices:
551         V_an_val = bs_call(S0, K, r, sigma, T)
552         V_mc_val, V_mc_err = mc_option_price(S0, K, r, sigma, T, N_paths)
553         V_fd_val = np.interp(S0, S_fd_check, V_fd_check)

```

```
554     print(f'{S0:>8.0f} {V_an_val:>12.2f} {V_mc_val:>12.2f} {V_fd_vall:>12.2f} {V_mc_err:>12.2f}')
555
556     print('\nAll figures saved to:', out_dir)
557     print('Done!')
558
559 if __name__ == "__main__":
560     main()
```